=== FILE: map_editor/ui/toolbar.py ===

```
"""Панель объектов — полностью автоматическая из конфигов"""

import os
import sys
import pygame
from ..config import COLORS, SYMBOL_COLORS, TEXTURES_DIR, NPC_DIR
from config.game_data import SYMBOLS_CONFIG, NPC_CONFIG

ROOT_DIR = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
class Toolbar:
    def __init__(self, rect):
        self.rect = rect
        self.items = []
        self.selected_index = 0
        self.scroll_offset = 0
        self.item_height = 48
        self.top_padding = 30

        self._build_items()

    def _build_items(self):
        """Автоматически собирает все объекты из конфигов"""
        self.items = []

        # ============================================================
        # 1. СТЕНЫ
        # ============================================================
        wall_symbols = []
        for symbol, config in SYMBOLS_CONFIG.items():
            if config.get('type') == 'wall':
                wall_symbols.append(symbol)
        wall_symbols.sort()

        self.items.append({'type': 'separator', 'label': 'СТЕНЫ'})
        for symbol in wall_symbols:
            surf = self._load_texture_or_color(symbol)
            self.items.append({
                'type': 'item',
                'symbol': symbol,
                'surface': surf,
                'label': symbol,
                'sub': 'стена'
            })

        # ============================================================
        # 2. ДВЕРИ
        # ============================================================
        door_symbols = []
        for symbol, config in SYMBOLS_CONFIG.items():
            if config.get('type') == 'door':
                door_symbols.append(symbol)

        if door_symbols:
            self.items.append({'type': 'separator', 'label': 'ДВЕРИ'})
            for symbol in door_symbols:
```

```
        surf = self._load_texture_or_color(symbol)
        door_type = SYMBOLS_CONFIG[symbol].get('door_type', 'normal')
        self.items.append({
            'type': 'item',
            'symbol': symbol,
            'surface': surf,
            'label': symbol,
            'sub': f'дверь {door_type}'
        })

# ============================================================
# 3. ОБЪЕКТЫ (спавн, выход, пол)
# ============================================================
self.items.append({'type': 'separator', 'label': 'ОБЪЕКТЫ'})

for symbol, config in SYMBOLS_CONFIG.items():
    if config.get('type') == 'player_spawn':
        surf = self._create_surface(symbol, (120, 100, 0))
        self.items.append({
            'type': 'item',
            'symbol': symbol,
            'surface': surf,
            'label': symbol,
            'sub': 'спавн'
        })
    elif config.get('type') == 'exit':
        surf = self._create_surface(symbol, (0, 100, 0))
        self.items.append({
            'type': 'item',
            'symbol': symbol,
            'surface': surf,
            'label': symbol,
            'sub': 'выход'
        })

surf = self._create_surface('_', (20, 20, 25))
self.items.append({
    'type': 'item',
    'symbol': '_',
    'surface': surf,
    'label': '_',
    'sub': 'пустота'
})

# ============================================================
# 4. ПРЕДМЕТЫ
# ============================================================
item_symbols = []
for symbol, config in SYMBOLS_CONFIG.items():
    if config.get('type') == 'item':
        item_symbols.append(symbol)

if item_symbols:
    self.items.append({'type': 'separator', 'label': 'ПРЕДМЕТЫ'})
    for symbol in item_symbols:
        surf = self._load_texture_or_color(symbol)
        config = SYMBOLS_CONFIG[symbol]
        item_type = config.get('item_type', '')
```

```
            if item_type == 'health':
                sub = 'аптечка +25 HP'
            elif item_type == 'armor':
                sub = 'броня +25 Armor'
            elif config.get('weapon_name'):
                sub = f'{config.get("weapon_name")} (+{config.get("ammo", 0)})'
            else:
                sub = 'предмет'

            self.items.append({
                'type': 'item',
                'symbol': symbol,
                'surface': surf,
                'label': symbol,
                'sub': sub
            })

    # ============================================================
    # 5. NPC
    # ============================================================
    self.items.append({'type': 'separator', 'label': 'NPC'})

    for symbol, config in NPC_CONFIG.items():
        name = config.get('name', 'unknown')
        surf = self._load_npc_sprite(name)
        if surf is None:
            surf = self._create_surface(symbol, (100, 0, 0))

        is_boss = config.get('class_name') == 'Boss'
        self.items.append({
            'type': 'item',
            'symbol': symbol,
            'surface': surf,
            'label': symbol,
            'sub': f'{name} {"(босс)" if is_boss else ""}'
        })

def _load_texture_or_color(self, symbol):
    """Загружает текстуру или создаёт цветной квадрат"""
    config = SYMBOLS_CONFIG.get(symbol, {})
    texture_path = config.get('texture') or config.get('sprite')

    if texture_path:
        full_path = os.path.join(ROOT_DIR, texture_path)
        if os.path.exists(full_path):
            try:
                surf = pygame.image.load(full_path).convert_alpha()
                size = 34
                return pygame.transform.scale(surf, (size, size))
            except:
                pass

    # Fallback: цвет
    color = SYMBOL_COLORS.get(symbol, (60, 60, 70))
    return self._create_surface(symbol, color)

def _load_npc_sprite(self, name):
    """Загружает спрайт NPC"""
    try:
```

```
            # Новая система: name_move_front_1.png
            path = os.path.join(NPC_DIR, name, f"{name}_move_front_1.png")
            if os.path.exists(path):
                surf = pygame.image.load(path).convert_alpha()
                size = 34
                return pygame.transform.scale(surf, (size, size))

            # Старая система: name_idle_front.png
            path_old = os.path.join(NPC_DIR, name, f"{name}_idle_front.png")
            if os.path.exists(path_old):
                surf = pygame.image.load(path_old).convert_alpha()
                size = 34
                return pygame.transform.scale(surf, (size, size))
        except:
            pass
        return None

    def _create_surface(self, symbol, color):
        size = 34
        surf = pygame.Surface((size, size))
        surf.fill(color)
        pygame.draw.rect(surf, (80, 80, 90), surf.get_rect(), 1)
        font = pygame.font.Font(None, 22)
        text = font.render(symbol, True, (255, 255, 255))
        text_rect = text.get_rect(center=surf.get_rect().center)
        surf.blit(text, text_rect)
        return surf

    def get_selected_symbol(self):
        if self.selected_index < len(self.items):
            item = self.items[self.selected_index]
            if item['type'] == 'item':
                return item['symbol']
        return 'M'

    def handle_click(self, mouse_x, mouse_y):
        if not self.rect.collidepoint(mouse_x, mouse_y):
            return False

        rel_y = mouse_y - self.rect.y - self.top_padding + self.scroll_offset
        index = rel_y // self.item_height

        if 0 <= index < len(self.items):
            self.selected_index = index
            return True

        return False

    def scroll(self, delta):
        total_height = len(self.items) * self.item_height + self.top_padding + 5
        max_scroll = max(0, total_height - self.rect.height + 10)
        self.scroll_offset = max(0, min(max_scroll, self.scroll_offset + delta))

    def draw(self, screen):
        pygame.draw.rect(screen, COLORS['panel_bg'], self.rect)
        pygame.draw.rect(screen, COLORS['panel_border'], self.rect, 1)

        # Заголовок
        font_title = pygame.font.Font(None, 16)
```

```
title = font_title.render("ОБЪЕКТЫ", True, (240, 240, 240))
screen.blit(title, (self.rect.x + 10, self.rect.y + 5))

# Счётчик
item_count = sum(1 for i in self.items if i['type'] == 'item')
font_count = pygame.font.Font(None, 11)
count_text = font_count.render(f"{item_count} объектов", True, (140, 140, 140))
screen.blit(count_text, (self.rect.x + 10, self.rect.y + 22))

# Список
y = self.rect.y + self.top_padding - self.scroll_offset
font_sep = pygame.font.Font(None, 13)
font_label = pygame.font.Font(None, 16)
font_sub = pygame.font.Font(None, 12)

for i, item in enumerate(self.items):
    if y + self.item_height < self.rect.y or y > self.rect.bottom:
        y += self.item_height
        continue

    rect = pygame.Rect(self.rect.x + 5, y, self.rect.width - 10, self.item_height)

    if item['type'] == 'separator':
        pygame.draw.rect(screen, (50, 50, 60), rect)
        text = font_sep.render(item['label'], True, (200, 200, 220))
        screen.blit(text, (rect.x + 10, rect.y + 14))
    else:
        if i == self.selected_index:
            pygame.draw.rect(screen, (80, 80, 160), rect)
        else:
            pygame.draw.rect(screen, (45, 45, 50), rect)

        if item['surface']:
            icon_rect = item['surface'].get_rect(topleft=(rect.x + 6, rect.y + 8))
            screen.blit(item['surface'], icon_rect)

        label_x = rect.x + 44
        label_y = rect.y + 6

        text = font_label.render(item['label'], True, (255, 255, 255))
        screen.blit(text, (label_x, label_y))

        sub_text = font_sub.render(item['sub'], True, (160, 160, 180))
        screen.blit(sub_text, (label_x, label_y + 22))

        pygame.draw.rect(screen, (70, 70, 80), rect, 1)

    y += self.item_height

# Полоса прокрутки
total_height = len(self.items) * self.item_height + self.top_padding + 5
if total_height > self.rect.height:
    bar_height = max(20, int(self.rect.height * (self.rect.height / total_height)))
    scroll_ratio = self.scroll_offset / (total_height - self.rect.height)
    bar_y = self.rect.y + int((self.rect.height - bar_height) * scroll_ratio)
    pygame.draw.rect(screen, (120, 120, 140), (self.rect.right - 10, bar_y, 6, bar_height))
```